

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Mon 9 February 2015 - Part Theory

NAME:

MATRICOLA:

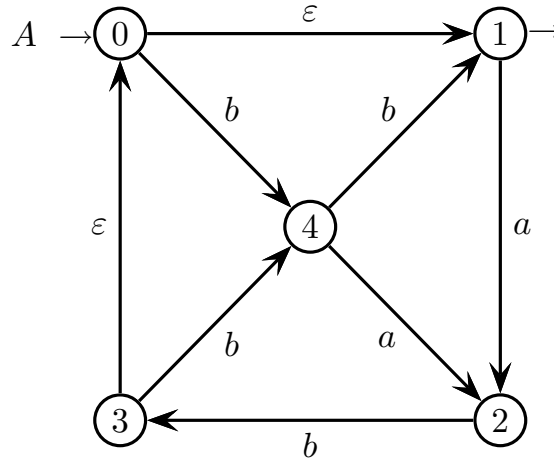
SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering also the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider the following nondeterministic finite state automaton A over the alphabet $\Sigma = \{a, b\}$, with spontaneous transitions (ε -transitions):



Answer the following questions:

- (a) By means of the Berry-Sethi method (BSM), find a deterministic automaton A' equivalent to A , and if necessary minimize the automaton A' and obtain the minimal automaton A'_{min} .
 - (b) By means of the node elimination method (Brzozowski) applied to the minimal automaton A'_{min} previously obtained, find a regular expression R equivalent to the automaton A .
 - (c) (optional) Cut the spontaneous transitions of automaton A , in such a way that the resulting automaton A'' is immediately deterministic (without any need to resort to BSM or to the subset construction after cutting).
-

2 Free Grammars and Pushdown Automata 20%

1. Write a non-extended (*BNF*) grammar for each of the following three languages L_1 , L_2 and L_3 , and draw the syntax trees of the strings listed below for each of them.

- (a) Language L_1 (axiom S):

$$L_1 = \{ (a \mid b)^n c^n \mid n \geq 1 \}$$

where:

$$s = a b a c c c$$

- (b) Language L_2 (axiom S):

$$L_2 = \{ a^m b^n c^p \mid m, n, p \geq 1 \wedge m + n = p \}$$

where:

$$t = a a b c c c$$

- (c) (optional) Language L_3 (axiom S):

$$L_3 = \{ a^m b^n c^p d^q \mid m, n, p, q \geq 1 \wedge m + n = p + q \}$$

where:

$$v_1 = a a b c c d \quad v_2 = a b b c d d \quad v_3 = a b b c c d$$

The grammar of language L_3 should not be ambiguous.

2. Consider a simplified version of the language of the first order logical formulas (predicate calculus), where a logical formula has the following syntactic features:

- a formula starts with a list, possibly empty, of existential and universal quantifiers, respectively denoted by “E” and “U”, each of which has one or more variables in its scope, separated by comma “,” (so called prenex form)
- a formula continues with a logical expression that may contain the logical operators sum “or”, product “and”, negation “not”, the round parentheses “(” and “)” to embrace logical subexpressions, and predicates (see below)
- the operator priority in a logical expression is as usual: logical negation (highest), logical product (middle), logical sum (lowest)
- a predicate is a name followed by a list, delimited by round parentheses, of one or more arguments separated by comma “,”
- a predicate argument is a variable
- a variable or a predicate name is an identifier, schematized by a terminal id

Sample formulas:

E x U y pred1 (x)

U alpha, beta E gamma pred1 (alpha) or pred2 (beta, gamma)

E x, y U p1, p2 (pred1 (x) or pred2 (p1, y)) and not pred2 (x, p2)

The language is a non-empty list of logical formulas as above. The formulas are separated by a semicolon “;” and the list is terminated by a dot “.”.

Answer the following questions:

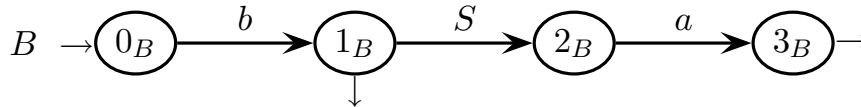
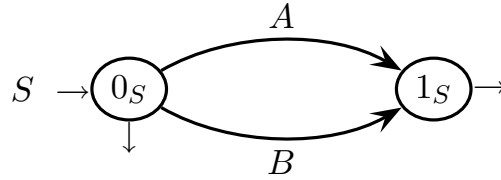
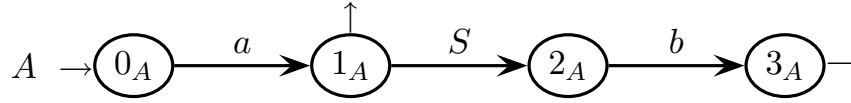
- (a) Write an extended grammar (*EBNF*), not ambiguous, that reasonably models the logical language sketched above.
- (b) (optional) We wish we extended the language and admitted also non-prenex formulas, where the quantifiers may appear in the middle, like:

U alpha E beta pred1 (alpha) or U gamma pred2 (beta, gamma)

Show which rules of the previous grammar should be modified to model the extended language. The new grammar should not be ambiguous either.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following machine net (where S is the axiom symbol):



Answer the following questions:

- (a) Compute the guide sets of the machine net (on the call arcs and exit darts), determine if the machine net is $ELL(1)$ and provide a suitable brief explanation.
- (b) Determine if the machine net is $ELR(1)$ by building the necessary portion of the pilot. If the $ELR(1)$ condition is not satisfied, list all the conflicts present in the produced (portion of the) pilot and briefly explain what conflict each of them is.
- (c) Consider the valid phrase “ ab ” and sketch its syntax tree (or trees if the phrase is ambiguous). Then execute the Earley syntax analysis of such a phrase and indicate the items in the Earley vector that contribute to acceptance (use the vector prepared on the next page).
- (d) (optional) Say if the grammar is ambiguous and briefly explain your answer.

Earley vector of string $a\ b$ (to be filled)
 (the number of rows is not significant)

0	a	1	b	2

Draw here the requested syntax tree(s)

4 Language Translation and Semantic Analysis 20%

1. Consider a source language of two-level lists with elements schematized by a , without separators, embraced by round parentheses “(” and “)”, like for instance:

$$(a \ a \ (a \ a) \ a \ (a))$$

We wish we had a one-valued translation τ that “flattens” such lists and reduces them to one-level only, as follows:

$$(a \ a \ (a \ a) \ a \ (a)) \xrightarrow{\tau} (a \ a \ 2a \ 2a \ a \ 2a)$$

by introducing the terminal “2” to mark the elements originally at level two. Consider the following non-extended (*BNF*) source grammar (axiom S), which generates the source language:

- 1: $S \rightarrow (L_1)$
- 2: $L_1 \rightarrow a L_1$
- 3: $L_1 \rightarrow a$
- 4: $L_1 \rightarrow (L_2) L_1$
- 5: $L_1 \rightarrow (L_2)$
- 6: $L_2 \rightarrow a L_2$
- 7: $L_2 \rightarrow a$

Answer the following questions:

- (a) Write a destination grammar associated to the above source grammar to compute the translation τ , and draw the syntax tree of the sample translation above. Say if the translation scheme for τ is deterministic and briefly explain your answer.
- (b) Consider the inverse translation τ^{-1} , say if it is deterministic and briefly explain your answer.
- (c) (optional) The source language (two-level lists) is well known to be regular. Write a rational translation expression R_τ that computes the translation τ .

2. The rules below (axiom S) define an abstract syntax for boolean expressions with the usual operators of logical product (**and**) and negation (**not**), as well as propositional letters schematized by terminal **a**:

$$1: S \rightarrow F$$

$$2: F \rightarrow F \text{ and } F$$

$$3: F \rightarrow \text{not } F$$

$$4: F \rightarrow a$$

As customary for an abstract syntax, the grammar above does not adequately represent the operator precedence, yet this fact is not relevant here. For instance, the sample boolean expression “a and not a and not a” has the syntax tree shown in the figure (next page).

- (a) Write an attribute grammar that computes, for each tree node of type F and for the tree root S , the number of propositional letters of the subtree rooted at that node, that in the overall tree, are in the scope of an even number of negations or of an odd number of negations. In the above example, two letters are in the scope of an even number of negations (first and third letter) and one letter is in the scope of an odd number of negations (second letter).

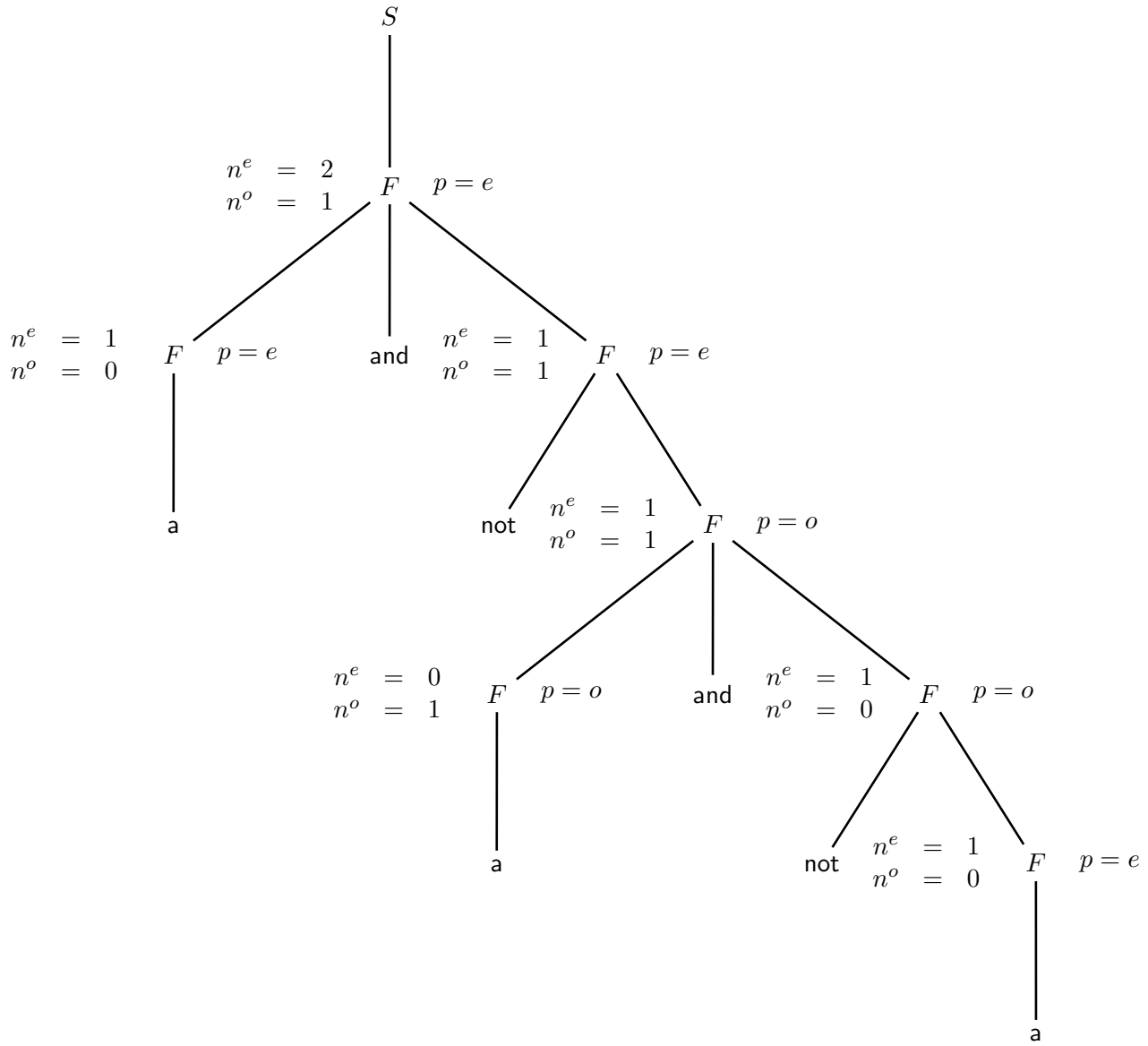
The grammar should use the following three attributes:

- $p \in \{e, o\}$ is a right attribute that indicates if the associated nonterminal is in the scope of an even or odd number of negations
- n^e and n^o are two left integer attributes that count the number of letters in the scope of an even or odd number of negations, respectively.

The values of these three attributes are exemplified in the tree figure (next page). See also the attribute table on the subsequent page.

- (b) (optional) Write a different attribute grammar that computes, in two attributes of the root node S that are called *even* and *odd*, the number of propositional letters that, in the entire expression, are in the scope of an even or of an odd number of negations. The grammar must use only left attributes. Notice that, if necessary, other grammar symbols besides S might have the attributes *even* and *odd*.

decorated tree for question *a*



attributes assigned to be used for question (a)

<i>type</i>	<i>name</i>	<i>domain</i>	<i>nonterm.</i>	<i>meaning</i>
right	p	$\{e, o\}$	F	parity of the number of negations F is in the scope of
left	n^e	integer ≥ 0	S, F	number of propositional letters that are in the scope of an even number of negations
left	n^o	integer ≥ 0	S, F	number of propositional letters that are in the scope of an odd number of negations

attributes to be completed or added for question (b)

<i>type</i>	<i>name</i>	<i>domain</i>	<i>nonterm.</i>	<i>meaning</i>
left	<i>even</i>	integer ≥ 0	$S \dots$	number of propositional letters that are in the scope of an even number of negations
left	<i>odd</i>	integer ≥ 0	$S \dots$	number of propositional letters that are in the scope of an odd number of negations

$$1: S_0 \rightarrow F_1$$

$$2: F_0 \rightarrow F_1 \text{ and } F_2$$

$$3: F_0 \rightarrow \text{not } F_1$$

$$4: F_0 \rightarrow \mathbf{a}$$

$$1: S_0 \rightarrow F_1$$

$$2: F_0 \rightarrow F_1 \text{ and } F_2$$

$$3: F_0 \rightarrow \text{not } F_1$$

$$4: F_0 \rightarrow \mathbf{a}$$

